Intro to Python Programming

We learn 5 concepts + 1 OOP

- Variable + Basic Calculation
- Data Types
- Data Structures
- Control Flow
- Function

```
print("hello world")
```

```
hello world
```

```
print("I am learning Python")
```

```
I am learning Python
```

```
print("this is another cell")
```

```
this is another cell
```

```
1
```

Variable & Calculation

```
print(1+1)
print(2*2)
```

```
2
4
```

```
print(100/2)
print(100//2)
```

```
50.0
50
```

```
3//2
```

```
1
```

```
# floor division
5 // 2
```

```
2
```

```
# modulo
5 % 2
```

```
1
```

```
5**2
```

```
25
```

```
5**3
```

```
125
```

```
5**4
```

```
625
```

```
## variable
x = 100
y = 200
print(x+y)
```

```
300
```

```
## naming variables
income = 50000
expense = 36000
saving = income - expense
print(saving)
```

```
14000
```

```
## snake case
income_y2025 = 65000

## delete variable
## del variable_name
```

```
1
```

Data Types

- int
- float
- str
- bool

```
## dynamic type
name = "Toy"
text = "I'm loving McDonalds"
age = 37 # int
gpa = 3.41 # float
```

```
fav_lang = "Python"
movie_lover = True # False

print(name, text, age, fav_lang, gpa, fav_lang, movie_lover)
```

```
Toy I'm loving McDonalds 37 Python 3.41 Python True
```

```
type(age)
```

```
int
```

```
## type hint
name: str = "Kasidis Toy"
type(name)
```

```
str
```

```
age: int = 37
gpa: float = 3.41
```

```
## convert type
str(100)

```

```
'100'
```

```
int(False)
```

```
0
```

```
# int() float() str() bool()
bool(0)
```

```
False
```

```
# f-string template
name = "toy"
age = 37
lang = "Python"

text = f"Hi! my name is {name} and I am {age} years old. My fav lang is {lang}!"
```

```
print(text)
```

```
Hi! my name is toy and I am 37 years old. My fav lang is Python!
```

```
1
```

Data Structures

- list
- tuple

- dictionary
- set

list is mutable

```
# shopping list
mom_list = ["egg", "milk", "bread", "banana", "rice"]
```

```
# check length of list
len(mom_list)
```

```
5
```

```
# index starts with 0
mom_list[0]
```

```
'milk'
```

```
# slice items in a list
mom_list[0:2]
```

```
['egg', 'milk']
```

```
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice']
```

```
## slice from bread till the end of list
mom_list[2: ]
```

```
['bread', 'banana', 'rice']
```

```
# list can contain multiple data types
toy = ["toy", 37, "bangkok", 3.41, True]
```

```
toy[0:2]
```

```
['toy', 37]
```

```
# nested list
brothers = [
    ["toy", 1988],
    ["top", 1989],
    ["tap", 1991]
]
```

```
brothers[1][1]
```

```
1989
```

```
brothers[2][0]
```

```
'tap'
```

```
## function vs. method
```

```
len(mom_list)
```

```
5
```

```
# list methods
# functions ที่สามารถใช้ได้กับ list เท่านั้น
```

```
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice']
```

```
# add carot at the end of list
mom_list.append("carot")
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice', 'carot']
```

```
# remove the last item
mom_list.pop()
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice']
```

```
# insert item at the index we want
mom_list.insert(1, "carot")
mom_list
```

```
['egg', 'carot', 'milk', 'bread', 'banana', 'rice']
```

```
# remove carot?
mom_list.remove("carot")
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice']
```

```
1
```

Mutable vs. Immutable

- mutable = change change/ transform e.g. `list` `dict`
- immutable = cannot change / transform e.g. `str` `tuple`

Function vs. Method

```
# function
print("hello world")
```

```
# method is a function for specific object (class)
object.method()
```

```
## string methods
text = "a duck walks into a bar."

```

```
text.upper()
```

```
'A DUCK WALKS INTO A BAR.'
```

```
text
```

```
'a duck walks into a bar.'
```

```
## list of my friends (mutable)
friends = ["lisa", "bambam", "jisoo"]
```

```
friends.append("kevin")
```

```
friends
```

```
['lisa', 'bambam', 'jisoo', 'kevin']
```

```
## string (immutable)
text = "a duck walks into a bar."
text = text.upper()
```

```
text
```

```
'A DUCK WALKS INTO A BAR.'
```

```
## lower case
text = text.lower()
```

```
text
```

```
'a duck walks into a bar.'
```

```
# new variable
lang = "Python"
lang[0] = "C"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-4267285996.py in <cell line: 0>()
      1 # new variable
      2 lang = "Python"
----> 3 lang[0] = "C"

TypeError: 'str' object does not support item assignment
```

```
new_lang = "C" + lang[1:]
```

```
new_lang
```

```
'Cython'
```

```
## update item in mom_list
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice']
```

```
mom_list.append("orange juice")
```

```
mom_list
```

```
['egg', 'milk', 'bread', 'banana', 'rice', 'orange juice']
```

```
## mutable = can change/ transform data
mom_list[0] = "duck egg"
mom_list[1] = "sheep milk"

mom_list
```

```
['duck egg', 'sheep milk', 'bread', 'banana', 'rice', 'orange juice']
```

```
## string methods
text
```

```
'a duck walks into a bar.'
```

```
text = text.replace("duck", "chicken")
```

```
text
```

```
'a chicken walks into a bar.'
```

```
text.count("chicken")
```

```
1
```

```
text = "...............hello......"
text = text.strip(".")
```

```
text
```

```
'hello'
```

```
# split text into list
text = "I love learning Python with P Toy"
split_text = text.split(" ")
```

```
" ".join(split_text)
```

```
'I love learning Python with P Toy'
```

```
1
```

tuple is immutable

```
my_tup = ()
```

```
type(my_tup)
```

```
tuple
```

```
type(mom_list)
```

```
list
```

```
books = ("make better decisions", "life's missing manual", "brain audit")
```

```
books[0]
```

```
'make better decisions'
```

```
books[1]
```

```
'life's missing manual'
```

```
books[2]
```

```
'brain audit'
```

```
books[0] = "Critical Thinking"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-961005094.py in <cell line: 0>()
----> 1 books[0] = "Critical Thinking"

TypeError: 'tuple' object does not support item assignment
```

```
books
```

```
('make better decisions', "life's missing manual", 'brain audit')
```

```
## tuple unpacking
x, y, z = (100, 200, 300)

print(x+y+z)
```

```
600
```

```
1
```

dictionary: mutable

```
{key: value}
```

```
movie = {
    "title": "The Dark Knight",
    "year": 2006,
    "rating": {
        "imdb": 8.6,
        "rottentomatoes": 9.1
    },
    "cast": ["A", "B", "C"]
}
```

```
movie
```

```
{'title': 'The Dark Knight',
 'year': 2006,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['A', 'B', 'C']}
```

```
movie["title"] = "Superwoman"
movie["year"] = 2026
```

```
movie
```

```
{'title': 'Superwoman',
 'year': 2026,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['A', 'B', 'C']}
```

```
movie["revenue"] = 1000
```

```
movie
```

```
{'title': 'Superwoman',
 'year': 2026,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['A', 'B', 'C'],
 'revenue': 1000}
```

```
del movie["revenue"]
```

```
movie
```

```
{'title': 'Superwoman',
 'year': 2026,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['A', 'B', 'C']}
```

```
type(movie)
```

```
dict
```

```
## dictionary methods
## keys, items, values
movie.keys()
```

```
dict_keys(['title', 'year', 'rating', 'cast'])
```

```
movie.values()
```

```
dict_values(['Superwoman', 2026, {'imdb': 8.6, 'rottentomatoes': 9.1}, ['A', 'B', 'C']])
```

```
movie.items()
```

```
dict_items([('title', 'Superwoman'), ('year', 2026), ('rating', {'imdb': 8.6, 'rottentomatoes':
9.1}), ('cast', ['A', 'B', 'C'])])
```

```
titles = list(movie.keys())
```

```
titles
```

```
['title', 'year', 'rating', 'cast']
```

```
## convert data structures
## list(), tuple(), dict(), set()
```

```
list_a = [100, 200, 300]
```

```
tuple(list_a)
```

```
(100, 200, 300)
```

```
## update value in dictionary
movie
```

```
{'title': 'Superwoman',
 'year': 2026,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['A', 'B', 'C']}
```

```
movie["cast"][0] = "Christian Bale"
```

```
movie
```

```
{'title': 'Superwoman',
 'year': 2026,
 'rating': {'imdb': 8.6, 'rottentomatoes': 9.1},
 'cast': ['Christian Bale', 'B', 'C']}
```

set: mutable

unique values in a set

```
colors = ["red", "blue", "blue", "red"]
```

```
set_colors = set(colors)
```

```
set_colors
```

```
{'blue', 'red'}
```

```
books = {"book1", "book2", "book3"}
type(books)
```

```
set
```

```
## add new item to set
set_colors.add("yellow")
set_colors.add("salmon")
```

```
set_colors
```

```
{'blue', 'red', 'salmon', 'yellow'}
```

```
## remove color
set_colors.remove('salmon')
```

```
set_colors
```

```
{'blue', 'red', 'yellow'}
```

```
1
```

control flow

- if
- for
- while

```
# if
score = 45

if score >= 80:
    print("passed")
else:
    print("failed")
```

```
failed
```

```
# multiple grades
score = 82

```

```
if score >= 80:
    print("A")
elif score >= 70:
    print("B")
elif score >= 60:
    print("C")
else:
    print("D")
```

```
A
```

```
# get input from user
username = input("what's your name: ")
age = int(input("how old are you: "))
```

```
what's your name: toy
how old are you: 37
```

```
# multiple conditions
# night club
has_id = True
age_over_18 = False
vip_customer = True

if has_id and (age_over_18 or vip_customer):
    print("Welcome to the club.")
else:
    print("Please come next time.")
```

```
Welcome to the club.
```

```
# for & while
items = ["egg", "milk", "bread"]

for item in items:
    if item == "milk":
        print("CP Meiji")
    else:
        print(item)
```

```
egg
CP Meiji
bread
```

```
## list comprehension
result = [ ]

for item in items:
    result.append(item.upper())

print(result)
```

```
['EGG', 'MILK', 'BREAD']
```

```
items
```

```
['egg', 'milk', 'bread']
```

```
# list comprehension
[item.upper() for item in items]
```

```
['EGG', 'MILK', 'BREAD']
```

```
## while
while True:
    user_input = input("Play on: Y/N ")
    if user_input == "N":
        print("Goodbye!")
        break
    else:
        print("Gooooo!")
```

```
Play on: Y/N Y
Gooooo!
Play on: Y/N Y
Gooooo!
Play on: Y/N Y
Gooooo!
Play on: Y/N N
Goodbye!
```

```
## while another example
count = 0
while count < 5:
    print("hi!")
    count += 1
```

```
hi!
hi!
hi!
hi!
hi!
```

```
1
```

Function

```
def function_name():
    pass
```

```
## creat our first function
def greeting(name):
    """
    say hello to our friends
    """
    print(f"hello {name}!")
```

```
greeting("toy")
```

```
hello toy!
```

```
## function can take inputs
greeting("jane")
```

```
hello jane!
```

```
## new function cube()
def cube(base, power):
    return base**power

result = cube(5, 4) ## 625

print(result)
```

```
625
```

```
## function can return multiple values
## using tuple unpacking
def random_fun(base):
    return base**2, base**3, base**4
```

```
x, y, z = random_fun(5)

print(x,y,z, x+y+z)
```

```
25 125 625 775
```

```
greeting("jisoo")
```

```
hello jisoo!
```

```
cube(10, 3)
```

```
1000
```

```
## function is "reusable"
```

```
## build my own chatbot
# def chat():
#     print("Hello!")
#     orders = {"hawaian": 0, "cheese": 0, "hotdog": 0}
#     bill = # total money (spending)
#     while True:
#         user_name = input("What's your name: ")
#         print(f"Welcome to our pizza restaurant Mr.{user_name}.")
#         ## another inputs ..

#         ## end the process
#         if user_input == "Check bill":
#             ## calculate total money spending
#             print("Thank you very much!")
#             break
```

```
## Recap all things we learn today
```

```
1
```

Five Essentials Concepts

1. Variable
2. Data Types
3. Data Structures
4. Control Flow
5. Function

```
1
```

OOP

Object Oriented Programming

```
class Employee():
    """
    A new employee at Google
    """
    def __init__(self, name, age, city):
        self.name = name
        self.age = age
        self.city = city

    def working(self):
        print("I'm working now!")
```

```
toy = Employee("Kasidis", 37, "Bangkok")
```

```
type(toy)
```

Employee
def __init__(name, age, city)

A new employee at Google

```
toy.name
```

'Kasidis'

```
# employee method
toy.working()
```

I'm working now!

```
1
```